

Prueba Técnica — QA Automation Lead (Ligo) V1.0

Ciclo E2E completo + explicación en video

¡Hola! Esta prueba busca ver cómo diseñas, automatizas, documentas y **explicas** un ciclo de pruebas de punta a punta. Nos importa más tu **criterio y la calidad de tus decisiones** que la cantidad de casos. Queremos entender cómo piensa y trabaja quien va a liderar la automatización de calidad en Ligo.

- **Tiempo estimado:** 5–6 horas.
- **Plazo de entrega:** 2 días corridos.
- **Entregas:** (1) un **repositorio público en GitHub** con todo el proyecto y (2) un **video** donde explicas tu trabajo (ver Parte 6, es obligatorio).

1. Qué vamos a evaluar

Área	Lo que queremos ver
Dominio de herramienta	Uso real y solvente de Playwright, Cypress, Karate o Appium (eliges una)
Patrones de automatización	Aplicación correcta de POM y/o otros patrones (Screenplay, fixtures, factories)
Metodología y buenas prácticas	Estructura, independencia de tests, manejo de esperas, localizadores robustos, datos de prueba
Diseño de casos	Cómo defines casos: técnicas, priorización, criterios de aceptación, trazabilidad
Ciclo E2E	Del diseño → implementación → ejecución → reporte, como un ciclo completo
Generación de evidencias	Reportes, screenshots, video/trace y logs que permitan auditar cómo se probó
Criterio técnico	Cómo justificas tus decisiones y trade-offs (hay puntos de decisión a propósito)

Área	Lo que queremos ver
Comunicación	Tu capacidad de explicar el trabajo con precisión (el video)

2. Elige tu track

Trabaja sobre **un** sistema público y estable, según la herramienta en la que seas más fuerte. No necesitas cuenta de pago para ninguno.

Track	Herramienta	Sistema objetivo (SUT) sugerido	URL
Web E2E	Playwright o Cypress	SauceDemo (e-commerce) o ParaBank (banca, web + API)	saucedemo.com / parabank.parasoft.com/parabank
API	Karate	restful-booker (CRUD + auth) o la API REST de ParaBank	restful-booker.herokuapp.com
Mobile	Appium	Sauce Labs My Demo App (APK Android)	github.com/saucelabs/sample-app-mobile/releases

Puedes **traer tu propio SUT** si crees que muestra mejor tu nivel. Si lo haces, justifica brevemente por qué lo elegiste: esa decisión también es parte de lo que evaluamos.

Datos útiles de SauceDemo: contraseña **secret_sauce** para todos los usuarios; existen usuarios con comportamientos especiales (**standard_user**, **locked_out_user**, **problem_user**, **performance_glitch_user**). Qué haces con ellos nos dice mucho de tu criterio.

3. El caso — un ciclo E2E

Automatiza un **flujo de usuario de punta a punta** que recorra el corazón del sistema. Sea cual sea tu track, el flujo debe cubrir estas historias (adáptalas al SUT que elijas):

- **HU-1 — Autenticación:** un usuario inicia sesión / se registra. Incluye al menos un camino feliz y uno de error (credenciales inválidas, usuario bloqueado, etc.).
- **HU-2 — Operación principal:** el usuario ejecuta la transacción central del sistema (en e-commerce: buscar producto → agregar al carrito → checkout; en banca: abrir cuenta → transferir fondos; en API: crear/actualizar/eliminar un recurso con su token).
- **HU-3 — Verificación de estado:** el sistema refleja correctamente el resultado (orden creada / saldo actualizado / recurso persistido) y el usuario puede consultarlo (historial, detalle, GET del recurso).
- **HU-4 — Casos negativos y de borde:** aplica partición de equivalencia y valores límite sobre al menos un campo crítico (montos, cantidades, campos obligatorios), y valida el manejo de errores.

Para cada historia define **criterios de aceptación** claros antes de automatizar.

4. Entregables obligatorios

4.1 Documento de diseño de casos de prueba

Un documento ([docs/casos-de-prueba.md](#) o una planilla) que incluya, por cada caso:

- ID, título y la historia/criterio que cubre.
- **Técnica de diseño** aplicada (partición de equivalencia, valores límite, tabla de decisión, transición de estados, etc.).
- **Prioridad y severidad** (justificadas).
- Precondiciones, datos de prueba, pasos y **resultado esperado**.
- Una **matriz de trazabilidad**: Historia de usuario → Caso de prueba → Test automatizado. Queremos ver que nada quede huérfano y que sabes qué cubre cada test.

4.2 Suite automatizada (en el repo)

- Implementada con tu herramienta elegida, aplicando **POM** (u otro patrón, justificado).
- Estructura clara, reutilización, **independencia entre tests** (que se puedan correr en cualquier orden y más de una vez).

- Manejo correcto de **esperas** (nada de **sleep** fijos), localizadores robustos, datos externalizados, configuración por ambiente.
- Etiquetado de suites (p. ej. **smoke** vs **regression**).

4.3 Generación de evidencias

Esto es central: queremos poder **revisar cómo se hicieron las pruebas** sin ejecutarlas nosotros.

- Reporte de ejecución legible (Playwright HTML report, Allure, Karate report, Mochawesome, etc.).
- **Screenshots** y **video o trace** de la ejecución (especialmente en fallos).
- Logs de la corrida.
- Deja un ejemplo de evidencia **ya generada** versionada en el repo (carpeta **evidencias/** o link al reporte), además de explicar cómo regenerarla.

4.4 README ejecutable

Para que **nosotros mismos** podamos clonar y correr todo:

- Prerrequisitos y versiones.
- Cómo instalar, cómo correr la suite, cómo generar el reporte/evidencias.
- Qué decisiones de arquitectura tomaste y por qué.
- Alcance: qué automatizaste, qué dejaste fuera y por qué.

4.5 Pipeline CI (*bonus, opcional*)

Un workflow (p. ej. GitHub Actions) que corra la suite y publique el reporte como artefacto. Suma, no resta.

5. Puntos de decisión (queremos ver tu criterio)

No hay una única respuesta correcta. En el documento y/o el video, justifica cómo resolviste al menos **tres** de estos:

1. ¿Qué partes del flujo automatizaste y cuáles **no**? ¿Por qué?
2. ¿Cómo preparaste los **datos de prueba** y el estado? (¿Setup vía API y validación por UI? ¿Datos efímeros? ¿Usuario fijo?)
3. ¿Cómo garantizas que la suite **no sea flaky** al correr en CI?
4. ¿Qué estrategia de **localizadores** usaste y por qué (rol/test-id vs. xpath)?

5. Si elegiste SauceDemo: ¿qué hiciste con `problem_user` / `performance_glitch_user`? Si elegiste un SUT con bugs (restful-booker, ParaBank): ¿reportaste las discrepancias que encontraste?
 6. ¿Qué dejarías para una **v2** de esta suite con más tiempo?
-

6. El video (obligatorio)

Graba un recorrido de **8 a 15 minutos** (Loom, o cualquier grabador de pantalla; deja el link en el README) donde **expliques con precisión cómo hiciste la prueba**. Sugerencia de guion:

1. **Enfoque (1–2 min)**: qué herramienta y SUT elegiste y **por qué**.
2. **Diseño (2–3 min)**: recorre tus casos de prueba y la matriz de trazabilidad — cómo decidiste qué probar.
3. **Código (3–4 min)**: tour del proyecto mostrando el patrón (POM), la estructura y 2–3 buenas prácticas concretas.
4. **Ejecución en vivo (2–3 min)**: corre la suite y muestra las **evidencias** que se generan (reporte, screenshots, trace/video).
5. **Decisiones (2 min)**: explica 2–3 decisiones técnicas y sus trade-offs (de la Parte 5).
6. **Mejoras (1 min)**: qué harías distinto o agregarías en una v2.

El objetivo del video es que podamos entender tu razonamiento y confirmar que **tú** hiciste y entiendes el trabajo. Habla con naturalidad; no buscamos una producción perfecta, buscamos claridad.

7. Checklist de entrega

- ☐ Repo público en GitHub con la suite, el README y las evidencias de ejemplo.
- ☐ Documento de diseño de casos + matriz de trazabilidad.
- ☐ Suite que corre en cualquier orden y genera reporte/evidencias.
- ☐ Link al video en el README.
- ☐ (Opcional) Pipeline CI.

Cualquier duda sobre el enunciado, escríbenos. Si por tiempo dejas algo fuera, dilo y explica qué harías con más tiempo: preferimos un alcance honesto y bien hecho a uno amplio y frágil. ¡Éxitos!